Методы понижения размерности

Машинное обучение

Наиль Баширов

Московский физико-технический институт

26 октября 2026



Сингулярное разложение

Определение

Сингулярное разложение матрицы $A \in \mathbb{R}^{n \times d}$ имеет вид

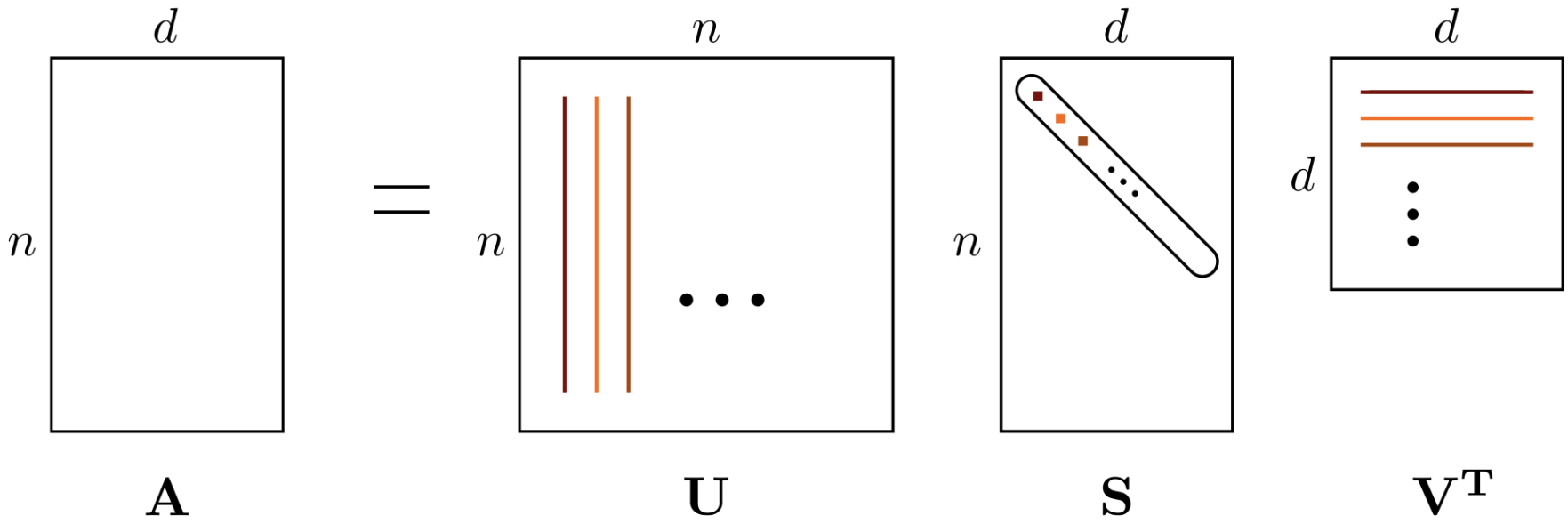
$$A = U \Sigma V^\top,$$

где

- матрица U размера $n \times n$ ортогональна,
- матрица V размера $d \times d$ ортогональна,
- матрица Σ размера $n \times d$ диагональна: $\Sigma_{ii} = \sigma_i \geq 0$.

Ненулевые сингулярные значения упорядочены как $\sigma_1 \geq \cdots \geq \sigma_r > 0$, где $r = \mathrm{rank}(A)$.

Сингулярное разложение



От собственных векторов к SVD

Пусть $A \in \mathbb{R}^{n \times m}$. Матрицы $A^\top A$ и $AA^\top$ имеют размеры $m \times m$ и $n \times n$ соответственно. Рассмотрим собственный вектор v_i матрицы $A^\top A$ с $\lambda_i > 0$:

Связь с собственным разложением

$$A^\top A v_i = \lambda_i v_i, \qquad \|v_i\| = 1, \sigma_i = \sqrt{\lambda_i}, \qquad u_i = \frac{A v_i}{\sigma_i}.$$

v_i и u_i называют правым и левым сингулярными векторами. Из $AA^\top u_i = \lambda_i u_i$ аналогично следует $v_i = A^\top u_i / \sigma_i$.

Геометрический смысл

A переводит v_i в u_i и растягивает его в σ_i раз. При $\sigma_i = 0$ полный U дополняют до ортонормированного базиса.

SVD даёт лучшее приближение низкого ранга

Оставим k наибольших сингулярных значений и соответствующие векторы. Теорема Эккарта-Юнга утверждает, что SVD является лучшим приближением в Фробениусовой норме:

Лучшее приближение ранга k

$$A_k = U_k \Sigma_k V_k^\top, \qquad A_k \in \operatorname*{argmin}_{\operatorname{rank}(B) \le k} \|A - B\|_F^2, \qquad \|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2.$$

Рандомизированное усечённое SVD

Если нужны только первые k компонент, не обязательно вычислять полное SVD. Случайная выборка помогает найти подпространство, которое хорошо приближает столбцы A. Рассмотрим алгоритм:

- Сгенерировать случайную матрицу $\Omega \in \mathbb{R}^{m \times \ell}$, где ℓ немного больше k, и вычислить $Y = A\Omega$;
- Найти ортонормированный базис Q столбцов Y с помощью QR-разложения;
- Вычислить $B = Q^\top A$ и его компактное SVD: $B = \widetilde{U}\Sigma V^\top$;
- Оставить первые k компонент: $A \approx Q\widetilde{U}_k\Sigma_k V_k^\top$.

Почему это работает?

Матрица $QQ^\top A$ является проекцией на найденное подпространство. Вместо A мы раскладываем маленькую матрицу $B = Q^\top A$.

Приложения SVD

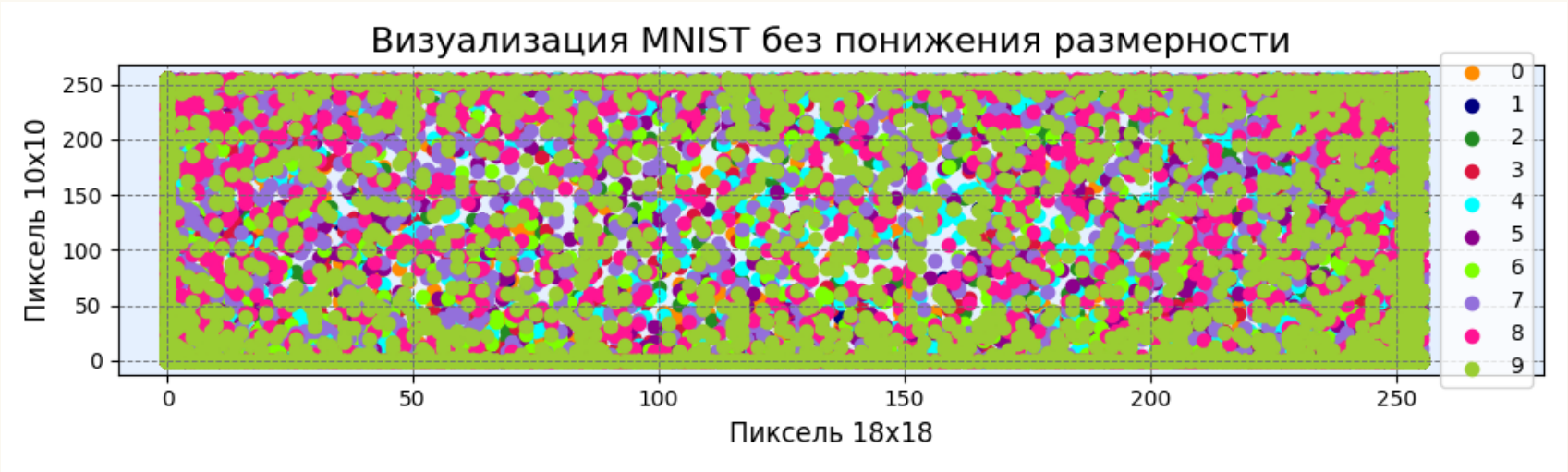
- Решение обратных задач и вычисление псевдообратной матрицы;
- Понижение размерности: метод главных компонент (PCA);
- Латентно-семантический анализ (LSA) в обработке естественного языка (NLP);
- Сжатие изображений;
- Подавление шума в обработке сигналов;
- Рекомендательные системы;

Мотивация

Задача: распознать цифры на картинке.



В MNIST каждая чёрно-белая картинка имеет размер 28×28. Представим её вектором из 784 пикселей. Тогда один объект становится точкой в $\mathbb{R}^{784}$.



Постановка задачи снижения размерности данных

Рассмотрим выборку $x^1, \ldots, x^n \in \mathbb{R}^d$ и представим её в $k < d$ измерениях. Сначала решим, какие свойства хотим сохранить:

- PCA: точность линейного восстановления;
- MDS: попарные расстояния;
- методы соседства: локальных соседей.

Зачем уменьшать размерность?

- сократить вычисления и объём хранения;
- убрать избыточность и упростить модель;
- показать данные на плоскости или в пространстве.

Метод главных компонент (PCA)

Пусть $X \in \mathbb{R}^{n \times d}$ задаёт матрицу объектов. Центрируем каждый признак и получаем X_c. Ищем $k < d$ ортонормированных направлений $V \in \mathbb{R}^{d \times k}$:

Проекция и восстановление

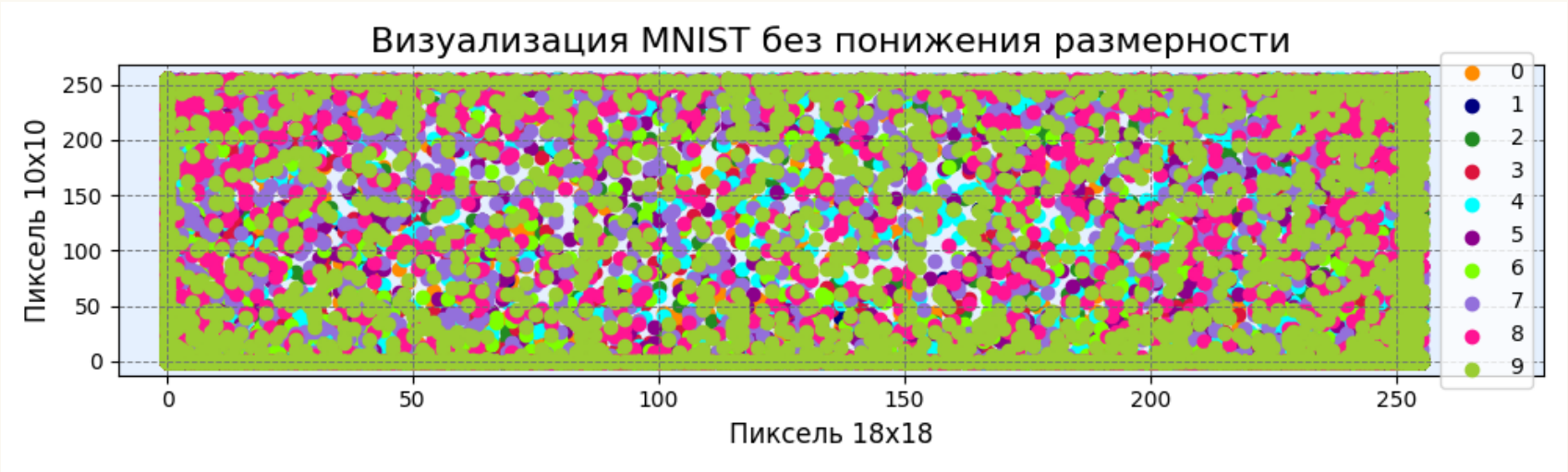
$$V^\top V = I_k, \qquad Z = X_c V, \qquad \widehat{X} = X_c V V^\top = Z V^\top.$$

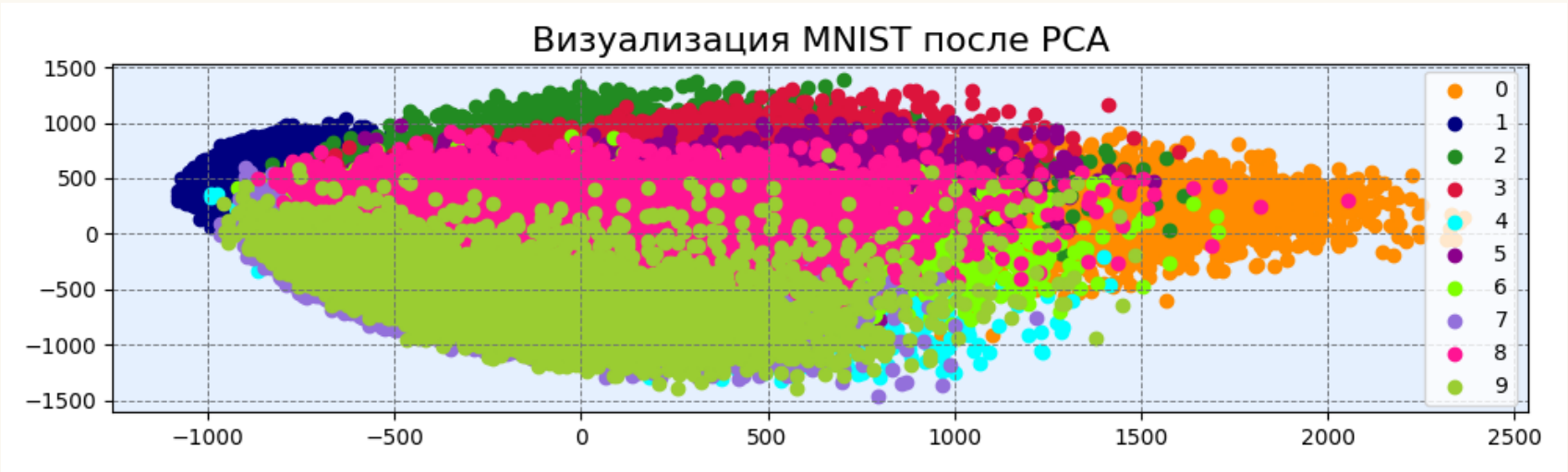
Строки $Z \in \mathbb{R}^{n \times k}$ содержат координаты объектов в новом пространстве. Выбираем направления, которые дают наименьшую ошибку восстановления:

$$\|X_c - Z V^\top\|_F^2 \longrightarrow \min_{V^\top V = I_k}.$$

Идея: найти плоскость, на которую данные проецируются с наименьшей среднеквадратичной ошибкой.

PCA на MNIST


Визуализация MNIST без понижения размерности
Пиксель 10x10
Пиксель 18x18
0
50
100
150
200
250
0
1
2
3
4
5
6
7
8
9


Визуализация MNIST после PCA
1500
1000
500
0
−500
−1000
−1500
−1000
−500
0
500
1000
1500
2000
2500
0
1
2
3
4
5
6
7
8
9

Почему PCA выбирает собственные векторы?

При фиксированных ортонормированных направлениях V оптимальные координаты равны $Z = X_c V$. Тогда ошибка равна

$$\|X_c - X_c V V^\top\|_F^2 = \|X_c\|_F^2 - \operatorname{tr}(V^\top X_c^\top X_c V).$$

Первое слагаемое постоянно. Поэтому минимизация ошибки равносильна максимизации:

Критерий PCA

$$\operatorname{tr}(V^\top C V), \qquad C = X_c^\top X_c, \qquad V^\top V = I_k.$$

По теореме Рэлея–Ритца столбцы V являются собственными векторами C с наибольшими собственными значениями: PCA оставляет направления с наибольшим разбросом.

Свойства PCA и ошибка восстановления

Обозначим через $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ собственные значения $C = X_c^\top X_c$. Пусть V содержит первые k собственных векторов. Тогда

$$V^\top V = I_k, \qquad Z = X_c V, \qquad Z^\top Z = \Lambda_k.$$

Столбцы Z ортогональны; норма j-го столбца равна $\sqrt{\lambda_j}$.

Остаточная ошибка

$$\|X_c - ZV^\top\|_F^2 = \|X_c\|_F^2 - \sum_{j=1}^{k} \lambda_j = \sum_{j=k+1}^{d} \lambda_j.$$

Восстановительная ошибка равна сумме отброшенных собственных значений. Для ковариационной матрицы отличается только общий множитель n или $n-1$.

Выбор количества признаков

Доля оставшейся ошибки после выбора первых k компонент:

Доля потери

$$E_k = \frac{\sum_{j=k+1}^{d} \lambda_j}{\sum_{j=1}^{d} \lambda_j}.$$

Чем меньше E_k, тем точнее восстановление исходных данных.

Выбор количества признаков

Доля оставшейся ошибки после выбора первых k компонент:

Доля потери

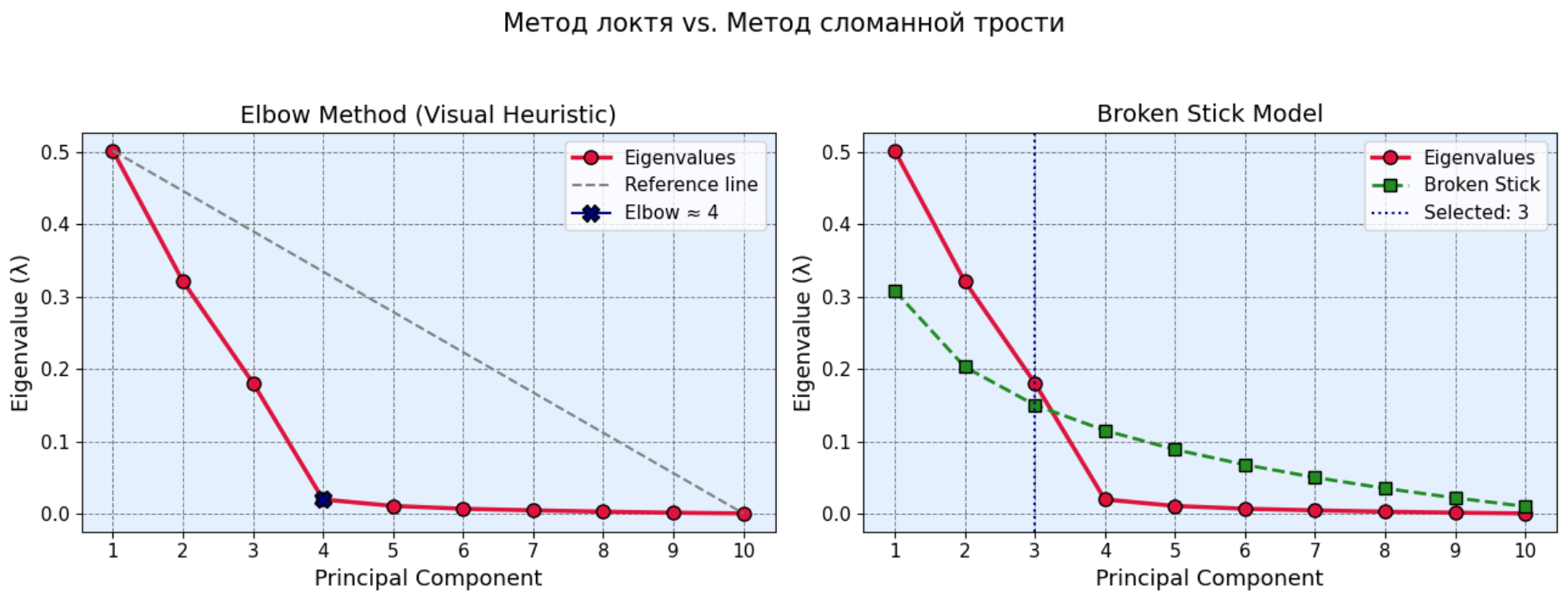
$$E_k = \frac{\sum_{j=k+1}^{d} \lambda_j}{\sum_{j=1}^{d} \lambda_j}.$$

Чем меньше E_k, тем точнее восстановление исходных данных.

- Задать порог ε и взять наименьшее k, для которого $E_k \leq \varepsilon$.
- **Метод локтя:** выбрать место, где спад λ_j становится пологим. Это визуальная эвристика.
- **Сломанная трость:** сравнить долю дисперсии компоненты с фоновым уровнем:

$$\frac{\lambda_j}{\sum_{\ell=1}^{d} \lambda_\ell} > \frac{1}{d} \sum_{\ell=j}^{d} \frac{1}{\ell}.$$

Метод локтя/метод сломанной трости



Главные компоненты: дисперсия проекций

Пусть $c_1, \ldots, c_d$ являются ортонормированными собственными векторами ковариационной матрицы с собственными значениями $\lambda_1 \geq \cdots \geq \lambda_d$.

- c_1 задаёт направление максимального разброса;
- c_j даёт максимальный оставшийся разброс среди направлений, ортогональных $c_1, \ldots, c_{j-1}$;
- координата объекта: $z_j^i = \langle x^i - \bar{x}, c_j \rangle$.

Геометрия

Первая компонента идёт вдоль самого длинного направления облака точек; каждая следующая по нормали описывает оставшийся разброс.

Особенности метода главных компонент

- Центрирование обязательно. Стандартизируйте признаки, если их единицы измерения должны иметь одинаковый вес.
- Выбросы могут повлиять на направления PCA; проверьте их природу, прежде чем удалять.
- Коррелированные признаки удалять не обязательно: PCA строит из них ортогональные комбинации.
- Поворот координат поворачивает и компоненты; изменение масштаба признаков меняет результат.
- При повторных собственных значениях отдельные направления не уникальны, но подпространство остаётся тем же.

Kernel PCA: PCA в пространстве признаков

Ядро $K(x, y) = \langle \varphi(x), \varphi(y) \rangle$ задаёт скалярные произведения после нелинейного отображения φ, не вычисляя его явно. Для $K_{ij} = K(x^i, x^j)$ центрируем матрицу:

$$K_c = HKH, \qquad H = I_n - \frac{1}{n} 1 1^\top.$$

Решаем задачу оптимизации и получаем координаты объектов:

Новые координаты

$$K_c q_j = \mu_j q_j, \qquad \|q_j\|_2 = 1; \quad z_j^i = \sqrt{\mu_j}\, q_{ij}.$$

Здесь PCA линейна в пространстве $\varphi(x)$, но нелинейна в исходных признаках.

MDS: сохранить заданные расстояния

Метрическое MDS ищет координаты $z^i \in \mathbb{R}^k$, сохраняющие заданные расстояния δ_{ij}, минимизируя stress:

$$\text{stress}(Z) = \sum_{i<j} w_{ij} \left(\|z^i - z^j\|_2 - \delta_{ij} \right)^2.$$

Для евклидовых расстояний классическое MDS восстанавливает координаты через матрицу $D^{(2)} = (\delta_{ij}^2)$ и центрирующую матрицу $H = I_n - \frac{1}{n} 1 1^\top$:

Классическое MDS

$$B = -\frac{1}{2} H D^{(2)} H = Q \Lambda Q^\top, \qquad Z = Q_k \Lambda_k^{1/2}.$$

Берут k наибольших положительных собственных значений; координаты определены с точностью до поворота и переноса.

Isomap: геодезические расстояния через граф

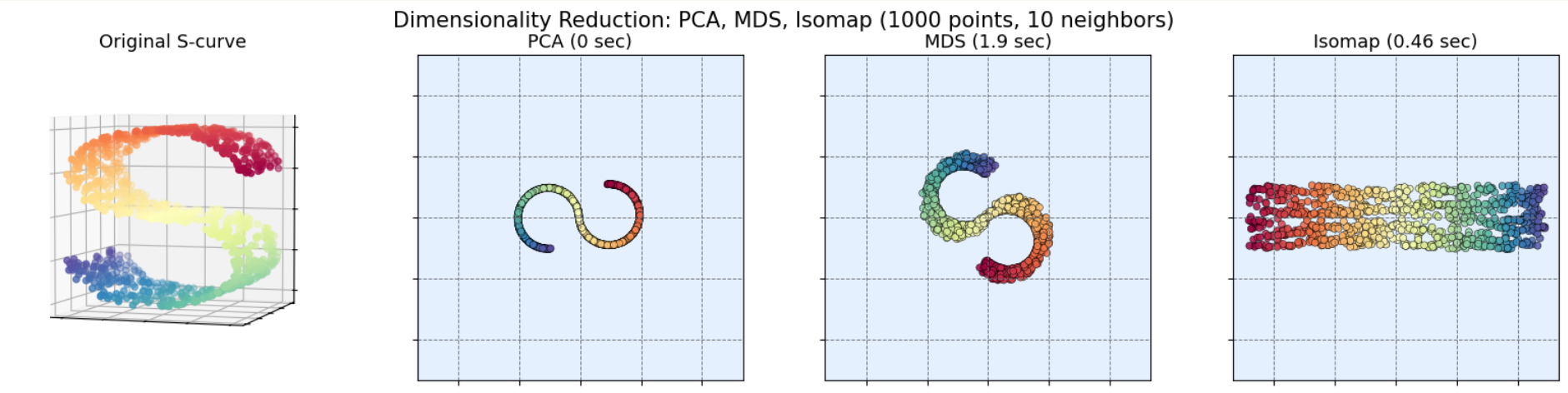
Три шага

1. Соединить каждую точку с ближайшими соседями и взвесить рёбра расстояниями.
2. Найти кратчайшие пути между всеми парами. Они приближают расстояния вдоль многообразия.
3. Применить классическое MDS к матрице этих расстояний.

Зачем нужен граф?

Прямая между точками может проходить сквозь «пустоту» между витками многообразия. Путь по графу следует вдоль поверхности; MDS затем укладывает эту геометрию в малую размерность.

Нелинейные методы понижения размерности



t-SNE

Главная идея

t-SNE (t-distributed Stochastic Neighbor Embedding) размещает точки так, чтобы в двумерной карте сохранились вероятности локального соседства.

Метод чаще всего используют для визуализации.

Что показывает карта t-SNE?

Сохраняем локальное соседство

Алгоритм старается оставить близких соседей рядом, но не подгоняет все расстояния между парами объектов.

SNE

В исходном пространстве вероятность соседства зависит от расстояния до x^i; масштаб σ_i задаётся отдельно для каждой точки:

Соседи в исходном пространстве

$$p_{j|i} = \frac{\exp\left(-\|x^j - x^i\|^2/2\sigma_i^2\right)}{\sum_{k\neq i}\exp\left(-\|x^i - x^k\|^2/2\sigma_i^2\right)}.$$

В низкой размерности вероятность соседства считаем аналогично:

Соседи на карте

$$q_{j|i} = \frac{\exp\left(-\|z^j - z^i\|^2\right)}{\sum_{k\neq i}\exp\left(-\|z^i - z^k\|^2\right)}.$$

Здесь $p_{i|i} = q_{i|i} = 0$; для каждой точки вероятности по всем соседям суммируются в 1.

Как выбирать σ_i^2

Энтропия распределения соседей точки x^i и соответствующая перплексия:

Перплексия: эффективное число соседей

$$H_i = -\sum_{j \neq i} p_{j|i} \log_2 p_{j|i}, \qquad \text{Perp}(x^i) = 2^{H_i}.$$

Задаём целевую перплексию. Для каждой точки бинарным поиском подбираем своё σ_i, чтобы учесть местную плотность данных.

Оптимизационная задача

Подбираем координаты $z^1, \ldots, z^n$, чтобы вероятности на карте $q_{j|i}$ были близки к исходным $p_{j|i}$.

Функция потерь

$$\mathcal{L}(P, Q) = \sum_{i=1}^{n} \mathrm{KL}(P_i \parallel Q_i) = \sum_{i=1}^{n} \sum_{j \neq i} p_{j|i} \log \frac{p_{j|i}}{q_{j|i}} \longrightarrow \min_Z .$$

В этой функции штрафы несимметричны:

- большой штраф: близкие в исходных данных точки оказались далеко;
- меньший штраф: далёкие точки случайно оказались рядом.

Симметричный SNE

В t-SNE условные вероятности объединяют в одно симметричное распределение по парам точек:

Вероятности пар

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \qquad q_{ij} = \frac{\exp\bigl(-\|z^i - z^j\|^2\bigr)}{\sum_{k \neq \ell} \exp(-\|z^k - z^\ell\|^2)}.$$

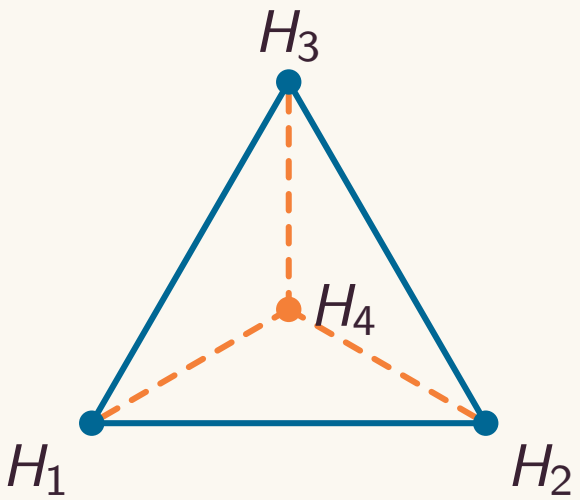
Здесь $p_{ii} = q_{ii} = 0$, $p_{ij} = p_{ji}$, $q_{ij} = q_{ji}$ и $\sum_{i \neq j} p_{ij} = \sum_{i \neq j} q_{ij} = 1$.
Функция потерь:

$$\mathcal{L}(P, Q) = \mathrm{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

Симметризация учитывает близость пары с обеих сторон; суммарная вероятность по всем парам равна единице.

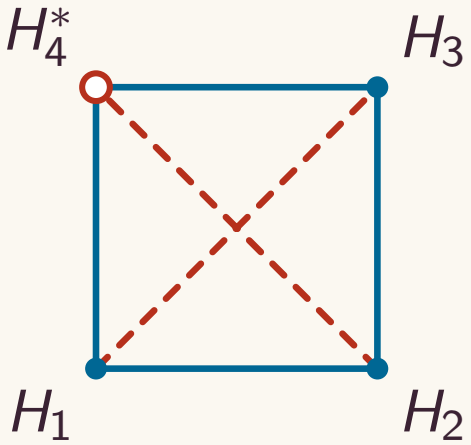
Crowding: все расстояния не уместить на плоскости

Тетраэдр в $\mathbb{R}^3$



Проекция схемы. Все шесть рёбер тетраэдра в 3D имеют длину d.

Один вариант на плоскости



В квадрате стороны равны d, диагонали равны $\sqrt{2}\,d$.

Зачем в t-SNE распределение Стьюдента?

На карте не- соседям нужно оставить место, чтобы близкие объекты не сжались в одну группу. Сравним веса пары на расстоянии r до нормировки:

Вес пары на карте

$$w_{\text{Gauss}}(r) = e^{-r^2}, \qquad w_{\text{Student}}(r) = \frac{1}{1+r^2} \sim \frac{1}{r^2}.$$

Почему это помогает?

У не-соседей исходная вероятность p_{ij} мала. При гауссовом весе далёкая пара также получает почти нулевое q_{ij} и почти не отталкивает точки. Вес Стьюдента убывает медленнее. Если после нормировки $q_{ij} > p_{ij}$, градиент раздвигает пару и освобождает место близким соседям. В t-SNE используют распределение Стьюдента с одной степенью свободы.

t-SNE: сходство на карте и градиент

Низкоразмерную вероятность соседства нормируем по всем парам:

Сходство на карте

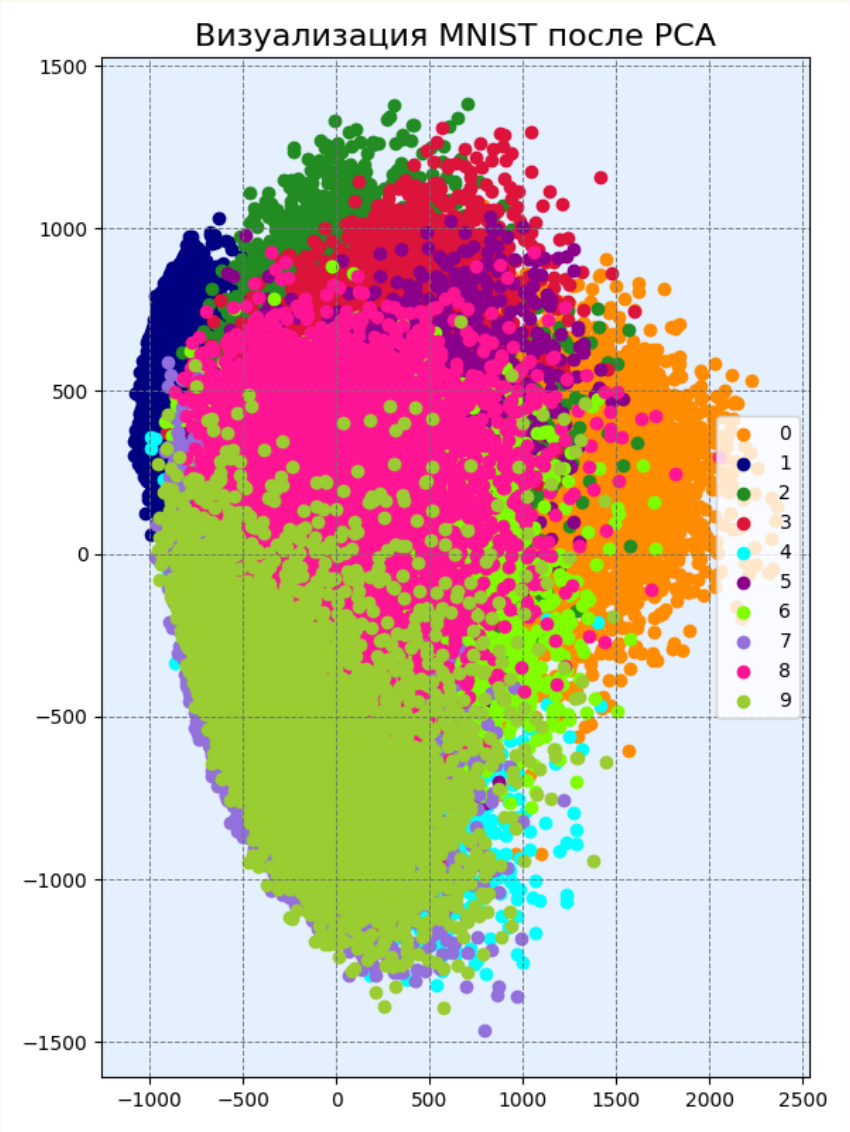
$$q_{ij} = \frac{(1 + \|z^i - z^j\|^2)^{-1}}{\sum_{k \neq \ell}(1 + \|z^k - z^\ell\|^2)^{-1}}.$$

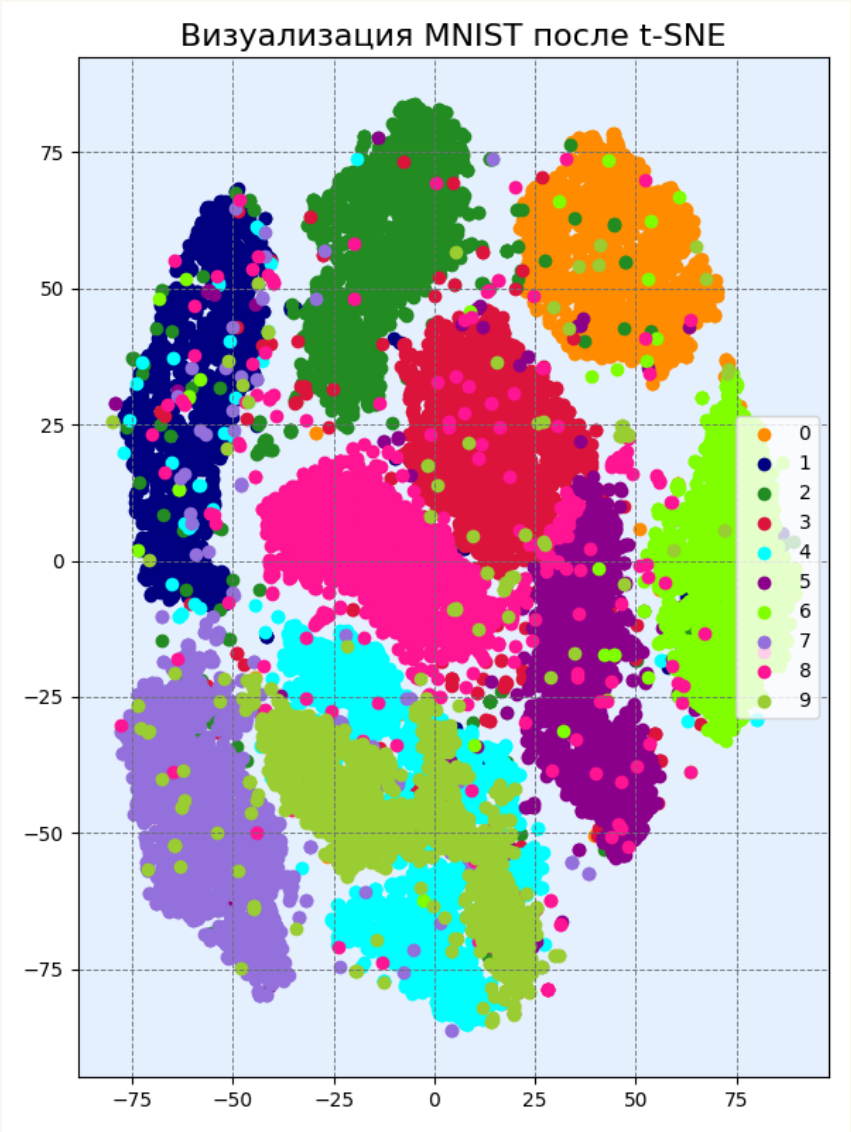
Для симметричной функции потерь градиент равен:

$$\frac{\partial \mathcal{L}}{\partial z^i} = 4 \sum_{j \neq i} \frac{(p_{ij} - q_{ij})(z^i - z^j)}{1 + \|z^i - z^j\|^2}.$$

- Если $p_{ij} > q_{ij}$, точки сближаются.
- Если $p_{ij} < q_{ij}$, точки отталкиваются.

Сравнение PCA и t-SNE на MNIST





UMAP

Главная идея

UMAP (Uniform Manifold Approximation and Projection) кодирует локальных соседей взвешенным графом, а затем строит похожий граф в пространстве меньшей размерности.

Ориентированный граф

Пусть $x^{i_1}, \ldots, x^{i_k}$ являются k ближайшими соседями точки x^i в метрике ρ. Обозначим расстояние до ближайшего соседа через $r_i = \min_s \rho(x^i, x^{i_s})$.

Вес направленного ребра

$$p_{i,i_s} = \exp\left(-\frac{\max\{0, \rho(x^i, x^{i_s}) - r_i\}}{\sigma_i}\right),$$

Масштаб σ_i подбирают так, чтобы $\sum_{s=1}^{k} p_{i,i_s} = \log_2 k$. Сдвиг на r_i даёт ближайшему соседу вес 1, а σ_i задаёт масштаб остальных связей.

Неориентированный граф

Объединим веса двух направлений по правилу fuzzy union:

Вес неориентированного ребра

$$p_{ij} = p_{i \to j} + p_{j \to i} - p_{i \to j} p_{j \to i}.$$

Это формула вероятности объединения двух независимых событий: сильной связи в любом из направлений достаточно, чтобы ребро получило большой вес.

Новые признаки: веса рёбер

Пусть $z^1, \ldots, z^n$ обозначают координаты объектов на карте. В новом графе вес ребра убывает с расстоянием:

Вес ребра на карте ($a, b > 0$)

$$q_{ij} = \frac{1}{1 + a\|z^i - z^j\|_2^{2b}}, \qquad 0 < q_{ij} \leq 1,$$

Вес p_{ij} задаёт силу связи исходной пары. Следующий шаг: сравнить p_{ij} и q_{ij} для каждой пары.

От KL к функции потерь UMAP

Зафиксируем пару (i, j). Обозначим наличие ребра через $E_{ij} \in \{0, 1\}$. В исходном графе $E_{ij} \sim \mathrm{Bern}(p_{ij})$, на карте $E_{ij} \sim \mathrm{Bern}(q_{ij})$.

Дивергенция Бернулли из KL

$$\mathrm{KL}(P_x \parallel P_z) = \sum_{e \in \{0,1\}} P_x(e) \log \frac{P_x(e)}{P_z(e)}$$

$$= p_{ij} \log \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \log \frac{1 - p_{ij}}{1 - q_{ij}}.$$

Обозначим $D_{ij} = \mathrm{KL}(\mathrm{Bern}(p_{ij}) \parallel \mathrm{Bern}(q_{ij}))$. Для карты минимизируем сумму по парам:

$$\mathcal{L}_{\mathrm{UMAP}} = \sum_{i<j} D_{ij} \longrightarrow \min_Z .$$

Большие p_{ij} сближают точки. Малые p_{ij} не дают несоседям слиться.

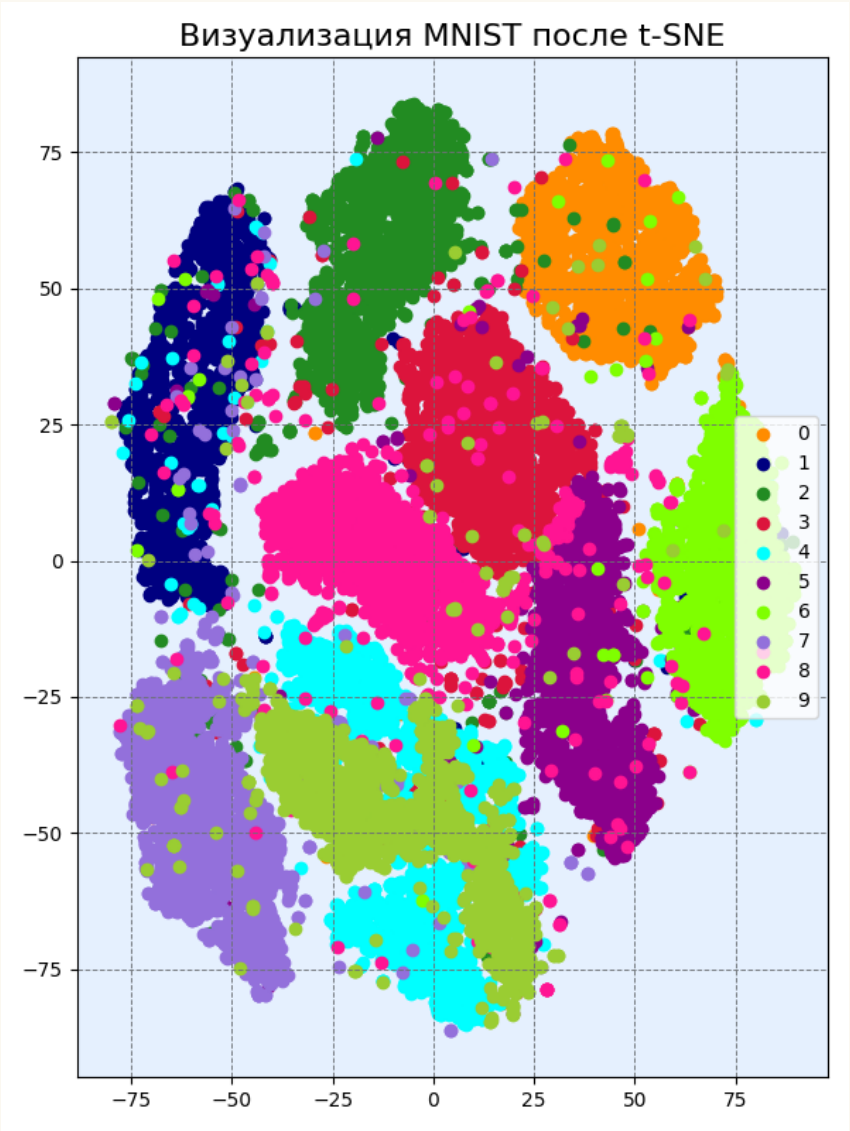
Свойства UMAP

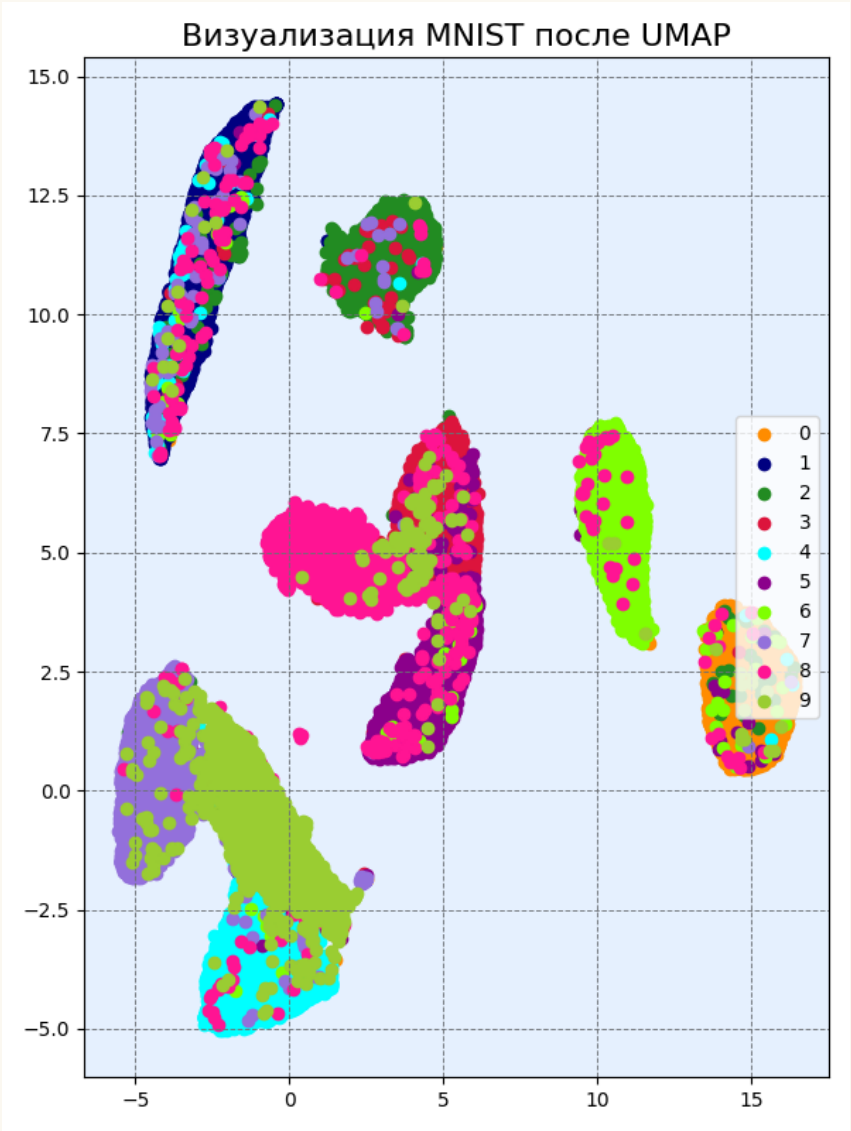
Что показывает карта

- Входной граф кодирует соседей, а не все исходные расстояния.
- Число соседей задаёт масштаб структуры, которую стараемся сохранить.
- Метрику выбирают с учётом типа данных.

Расстояния между далёкими кластерами не обязательно совпадают с исходными. Разреженный граф позволяет эффективно строить карту.

Сравнение t-SNE и UMAP на MNIST





Пример для привлечения внимания

4 миллиона транскриптомов отдельных клеток взрослого мозга мыши, помеченных по исходному региону мозга и представленных с помощью UMAP (Yao Z. et al., 2023, bioRxiv).

